Chapter II. Theoretical framework

1. Basic concepts

1.1. Functions

Let A and B be two arbitrary sets; $A \mapsto B$ denotes the set of all total functions from A to B. "$\mapsto$" associates to the right. We assume that all functions are either constants or have exactly one argument. This allows to write function application in "curried" form as $f\ x$. This "juxtaposition" associates to the left; i.e. $f\ x\ y$ means $(f\ x)y$. Functions with several arguments as $f \in A \times B \mapsto C$ are applied to tuples as in $f\langle x, y\rangle$. Alternatively, f can be defined for the isomorphic domain $A \mapsto (B \mapsto C)$ in which case we write $f\ x\ y$.

We use the semicolon (;) to break the normal precedence, i.e. $f; g\ h$ means $f(g\ h)$. Several terms separated by ; associate to the right, i.e. $T_1; T_2; T_3$ means $T_1(T_2(T_3))$.

New functions can be constructed from old ones by using composition, conditionals, and λ-abstraction as defined below.[1] This language is essentially Church's λ–calculus [Ch51]. However, we are not so much interested in the λ–calculus as a formal system; rather we use it as a notational vehicle.

[λ-abstraction] Let $T(x)$ be a term with the potentially free variable x, then $\lambda\ x.T(x)$ denotes a function f such that $f\ y = T(y)$.

The notations " **let** $x = T_1$ **in** T_2" and "T_2 **where** $x = T_1$" are both equivalent to $(\lambda x.T_2)T_1$.

Some special sets used are integer N and truthvalues $\{TT, FF\}$.

1.) We are not strictly formal here, rather we identify terms denoting functions with the function denoted. A formal treatment would require to introduce the semantics of the λ–calculus (e.g. [St77]).

[Conditionals] The conditional **if** T_1 **then** T_2 **else** T_3 has the usual meaning; it is only well defined if T_1 denotes an element in $\{TT, FF\}$. This definition of conditionals will be extended to domains in section 2.3.

[Redefinition] Let $f \in A \mapsto B$, then $f[x/y]$ denotes the function

$$\lambda\epsilon.\ \textbf{if}\ \epsilon = y\ \textbf{then}\ x\ \textbf{else}\ f\ \epsilon.$$

1.2. First order logic

We assume that the reader is familiar with basic concepts of logic (see e.g. Mendelson [Me64] or [Kl67]). The following defintion of first order logic is written in the style of denotational definitions and is mainly included here to give the reader an idea of the flavour of these definitions using a familiar example. Note, however, that all sets and functions are ordinary mathematical sets and total functions and do not have any added structure as they do in denotational semantics. This is possible since the domains in question are very simple and the problem of self application does not arise. Parentheses $[\![x]\!]$ are used to indicate that x is a syntactic object; this convention will be obeyed throughout this thesis.

1.2.1. Syntax

$x \in X$	infinite set of variables
$f_i \in Y_i$	i - ary function symbols
$p_i \in P_i$	i - ary predicate symbols
$t \in T = x \mid f_n(t_1, \ldots, t_n)$	Terms
$u \in F = true \mid false \mid t_1 = t_2 \mid p_n(t_1, \ldots, t_n) \mid u_1 \wedge u_2 \mid \neg u \mid \forall x.u$	Formulas

Terms $f_0()$ are abbreviated as f; if no ambiguities arise the arity index is omitted. Parentheses may be used arbitrarily to disambiguate. The formulas $u_1 \vee u_2$, $u_1 \Rightarrow u_2$, and $\exists x.u$ are abbreviations for $\neg(\neg u_1 \wedge \neg u_2)$, $\neg u_1 \vee u_2$, and $\neg(\forall x.\neg u)$ respectively. Some common function symbols like $+$, $-$, etc. are written infix.

1.2.2. Semantics

The semantics is defined by specifying the universe U of interpretations and "meaning" functions mapping terms into U and formulas into truth values. Formally we write:

U	Universe
$Fun_i = U \mapsto U \mapsto \ldots \mapsto U$	i - ary funtions
$Pre_i = U \mapsto \ldots \mapsto U \mapsto \{TT, FF\}$	i - ary predicates
$\phi \in \Phi = (X \mapsto U) \times (Y_i \mapsto Fun_i) \times (P_i \mapsto Pre_i)$	Interpretations

The last line, for example, is to be read as: "an interpretation ϕ is a function that maps variables into the universe U, i-ary function symbols into i-ary functions and i-ary predicate symbols into i-ary predicates.

$$\boxed{\mathcal{T} \in T \mapsto \Phi \mapsto U}$$

$$\begin{aligned}\mathcal{T}[\![x]\!]\phi &= \phi[\![x]\!] \\ \mathcal{T}[\![f(t_1,\ldots,t_n)]\!]\phi &= (\phi[\![f]\!])(\mathcal{T}[\![t_1]\!]\phi)\ldots(\mathcal{T}[\![t_n]\!]\phi)\end{aligned}$$

$$\boxed{\mathcal{F} \in F \mapsto \Phi \mapsto \{TT, FF\}}$$

$$\begin{aligned}\mathcal{F}[\![true]\!]\phi &= TT \\ \mathcal{F}[\![false]\!]\phi &= FF \\ \mathcal{F}[\![t_1 = t_2]\!]\phi &= (\mathcal{T}[\![t_1]\!]\phi = \mathcal{T}[\![t_2]\!]\phi) \\ \mathcal{F}[\![p(t_1,\ldots,t_n)]\!]\phi &= (\phi[\![p]\!])(\mathcal{T}[\![t_1]\!]\phi)\ldots(\mathcal{T}[\![t_n]\!]\phi) \\ \mathcal{F}[\![u_1 \wedge u_2]\!]\phi &= \textbf{if } \mathcal{F}[\![u_1]\!]\phi \textbf{ then } \mathcal{F}[\![u_2]\!]\phi \textbf{ else } FF \\ \mathcal{F}[\![\neg u]\!]\phi &= \textbf{if } \mathcal{F}[\![u]\!]\phi \textbf{ then } FF \textbf{ else } TT \\ \mathcal{F}[\![\forall x.u]\!]\phi &= (\lambda\epsilon.\mathcal{F}[\![u]\!]\phi[\epsilon/x] = \lambda\epsilon.TT)\end{aligned}$$

Other logical notions such as validity, satisfiability, proof, etc. can be defined in the obvious way in our formalism.

2. Scott's logic of computable functions

The mathematical theory underlying denotational semantics has been developed by Dana Scott [Sc70]. In lack of a better name we refer to this theory as Scott's logic of computable functions or Scottery for short. The theory we use differs slightly from that originally proposed by Scott; we assume an interpretation over complete partial orders (cpo's) [Pl78] rather than over lattices.

In this section we briefly summarize the relevant concepts and definitions of Scott's logic of computable functions. For a more complete treatment of this subject see [St77].

2.1. Basic definitions

[Quotients] Let S be a set and R be an equivalence relation on S, then the *quotient* S/R is the set of equivalence classes of R. If f is a function defined on S, then S/f is S/R where $R = \{(x, y) \mid f(x) = f(y)\}$.

[Directed set] If $(D, \sqsubseteq)$ is a partial order, then $S \subseteq D$ is *directed* if S has an upper bound.

[CPO] $(D, \sqsubseteq)$ is a *complete partial order* (*CPO*) if $(D, \sqsubseteq)$ is a partial order with

- bottom ($\bot_D \in D$) is the least element in D, and
- every directed subset S of D has a least upper bound ($\bigsqcup S$) in D.

[Domains] For the purpose of this thesis a domain is a CPO; we write D for a domain and omit the ordering $\sqsubseteq$ if it is clear from the context.[2]

Some domains have a set of *error elements* E; i.e. they are of the form $D + E$. We sometimes omit E in the definitions. Operations which are defined on D extend to $D + E$ in a strict way; that is they are the identity on E. If it is not necessary to distinguish different errors we use $E = \{?\}$.

[Flat Domains] Given an arbitrary set S the CPO $(S_\bot, \sqsubseteq)$ with $S_\bot = S \bigcup \{\bot\}$ and $a \sqsubseteq b$ iff $a = \bot$ is called a *Flat Domain*. Elements in S are called proper.

Flat domains are important because they allow us to construct a domain from a given set. Some typical examples we will use later are

$$T = \{TT, FF\}_\bot, \quad N = Z_\bot$$

where Z are the usual integers.

2.2. Operations on domains

[Sum] If D_1 and D_2 are two domains, then $D_1 + D_2$ is the *Sum* of D_1 and D_2. Elements of $D_1 + D_2$ are elements from $\{\langle 1, x\rangle \mid x \in D_1\} \bigcup \{\langle 2, x\rangle \mid x \in D_2\}$ with $\langle 1, \bot_{D_1}\rangle = \langle 2, \bot_{D_2}\rangle = \bot_{D_1+D_2}$. $\sqsubseteq$ extends to the sum according to $\langle i, x\rangle \sqsubseteq \langle j, y\rangle$ iff $i = j \wedge x \sqsubseteq y$.

2.) Topological issues such as finite basis are irrelevant for our discussion. For more details see [Sc72a, Sc76b, Pl78].

If $D = D_1 + D_2$ and $x \in D_i$, then $x : D$ denotes $\langle i, x\rangle$, the injection of x into D. For $z = \langle i, y\rangle \in D$ the predicate $z \mathrel{E} D_j$ is true iff $i = j$. If $z = \langle i, y\rangle \in D$, then $z \mid D_j$ is the projection of z onto the component D_j, i.e.

$$\langle i, y\rangle \mid D_i = \begin{cases} \bot & \text{if } i \neq j \\ y & \text{if } i = j \end{cases}$$

Frequently the explicit projections and injections are omitted if no ambiguities arise.

[Product] $D_1 \times D_2$ is the *Product* domain of D_1 and D_2 with $\langle x_1, x_2\rangle \sqsubseteq \langle y_1, y_2\rangle$ iff $x_1 \sqsubseteq y_1 \wedge x_2 \sqsubseteq y_2$. If $x \in D_1 \times D_2$ then $x = \langle x^{\#1}, x^{\#2}\rangle$. Note, while x_i is a variable, "superscript $\#i$" is a projection function defined on product domains.

Sum and product spaces extend to more than two dimensions in the obvious way.

[Continuity] A function $f \in D_1 \mapsto D_2$ is *continuous* if for all directed subsets $S \subseteq D_1$ we have $f(\bigsqcup S) = \bigsqcup f(S)$, where $f(S) = \{f(s) \mid s \in S\}$.

[Function domains] $D_1 \to D_2$ is the domain of continuous functions from D_1 into D_2 ordered by $f \sqsubseteq g$ iff $\forall x \in D_1 . f(x) \sqsubseteq g(x)$.[3]

[Monotonic] A function f is *monotonic* if $x \sqsubseteq y \to f(x) \sqsubseteq f(y)$.

Corollary: Every continuous function is monotonic. (Consider the directed set $S = \{x, y\}$ with $x \sqsubseteq y$.)

Corollary: If D_1 is a flat domain, then any monotonic $f \in D_1 \mapsto D_2$ is continuous. (All directed sets of a flat domain are trivial.)

[strict] A function $f \in D_1 \mapsto D_2$ is *strict* if $f(\bot_{D_1}) = \bot_{D_2}$.

Corollary: Any strict function defined on a flat domain is monotonic and continuous. (Immediate).

[Strict product] Let R be the relation on $D_1 \times D_2$ such that

$\langle x_1, x_2\rangle R \langle y_1, y_2\rangle$ iff $(x_1 = \bot_{D_1} \vee x_2 = \bot_{D_2}) \wedge (y_1 = \bot_{D_1} \vee y_2 = \bot_{D_2})$.

3.) Note the difference between $A \mapsto B$ and $A \to B$.

The domain $D_1 \times_s D_2 = (D_1 \times D_2)/R$ is the *Strict Product* of D_1 and D_2.

Arbitrary expressions over domains using the operators $+$, $\times$, $\rightarrow$ define new domains. We use the convention that $\rightarrow$ associates to the right. $+$ and $\times$ are associative (up to isomorphism).

[Reflexive domains] Recursive (mutually) domains equations define reflexive domains. Any recursive definition gives rise to a well defined domain as is shown in [Sc76b].[4]

As an example consider lists L over a domain of atoms A. The domain L can be defined recursively as $L = A + A \times L$. Let us briefly investigate the structure of L. Clearly, L contains the chain

$$\bot \sqsubseteq \langle a, \bot \rangle \sqsubseteq \langle a, \langle a, \bot \rangle \rangle \sqsubseteq \langle a, \langle a, \langle a, \bot \rangle \rangle \rangle \sqsubseteq \ldots .$$

For L to be a cpo it has to contain the limit of this chain which is an infinite list of a's. This is undesirable if, for example, we are only interested in finite lists as in LISP.

We get a different situation if we define $\acute{L} = A + A \times_s \acute{L}$, using the strict product. Now we have

$$\bot = \langle a, \bot \rangle = \langle a, \langle a, \bot \rangle \rangle = \langle a, \langle a, \langle a, \bot \rangle \rangle \rangle = \ldots ,$$

thus at least this sequence does not generate infinite objects. In fact, it can be shown that $\acute{L}$ contains exactly the finite lists over A together with $\bot$; $\acute{L}$ is the flat cpo of finite lists over A.

Another example for domains with infinite objects are binary trees B with $B = B \times B + A$. Again, B contains infinite objects. Using strict products gives us exactly the finite binary trees.

2.3. Notations

2.3.1. Conditionals

Conditionals **if** $-$ **then** $-$ **else** appearing in formulas of Scott's logic are defined completely strict in the condition.

$$\textbf{if } E_0 \textbf{ then } E_1 \textbf{ else } E_2 = \begin{cases} \bot & \text{if } E_0 = \bot \\ E_1 & \text{if } E_0 = \mathit{true} \\ E_2 & \text{if } E_0 = \mathit{false} \end{cases}$$

4.) The proof is based on a univeral domain $U = U \rightarrow U$, other domains are defined as images of U under retracts.

The **if** construct may appear without an else part in which case it is defined as:

$$\textbf{if } E_0 \textbf{ then } E_1 = \begin{cases} \bot & \text{if } E_0 = \bot \\ E_1 & \text{if } E_0 = \mathit{true} \\ \bot & \text{if } E_0 = \mathit{false} \end{cases}$$

Both versions of the conditional are continuous.

2.3.2. Lists

Finite lists (or sequences) over a given domain occur so frequently that we introduce a special operator. If D is a domain then D^* is defined as $D^* = D + D \times_s D^*$.

We write $\langle\rangle$ for the empty sequence. $\langle x_1, \ldots, x_n \rangle$ is a sequence with elements $x_1, \ldots, x_n$. The function *hd* and *tl* yield the head and tail of a sequence and $\bot$ if applied to $\langle\rangle$.

A centered dot "·" is used to denote concatenation. When no ambiguities arise we write $\langle x \rangle \cdot w$ as $x \cdot w$.

If s is a sequence $\mid s \mid$ denotes the length of s.

Let $f \in D_1 \to D_2$ be a function; unless otherwise stated $f^* \in D_1^* \to D_2^*$ is defined componentwise as

$$f^*\langle\rangle = \langle\rangle$$
$$f^*(d_1 \cdot \ldots \cdot d_n) = f(d_1) \cdot f(d_2) \cdot \ldots \cdot f(d_n)$$

2.4. Fixed points

[Fixed points] Let $f \in D \to D$, then $fix\ f$ denotes the least fixed point of f, that is the least element of $\{g \mid g = f(g)\}$.

For monotonic f the least fixed point exists; for continuous f the least fixed point is given by $fix\ f = \bigsqcup\{f_i\}$ where $f_0 = \bot$, $f_{i+1} = f(f_i)$.

2.4.1. Fixed point induction

Suppose we are given an object which is defined as the least fixed point of a function, say $f = fix\ h$. If we want to prove that a property P holds for f we can do this by showing $P(\bot)$ and $\forall x.P(x) \Rightarrow P(h\ x)$. This induction schema only holds if property P is "admissible".

[admissible] P is *admissible* if and only if $P(\bigsqcup f_i)$ whenever $P(f_i)$ for all directed sets $\{f_i\}$.

To demonstrate the effect of admissibility consider $g = fix(\lambda\, x.a{\cdot}x)$, an infinite list of a's. Consider the non-admissible proposition $P \equiv \lambda\, x.x \neq a{\cdot}x$. Clearly, $P(\perp)$. Assuming $P(x)$, i.e. $x \neq a{\cdot}x$ it follows $a{\cdot}x \neq a{\cdot}a{\cdot}x$ thus we have $P(a{\cdot}x)$. But observe that P is not true for the fixed point g since by definition of the fixed point we have $g = a{\cdot}g$.

2.4.2. Reasoning about fixed points

At several places we give manual proofs about properties of fixed points. Sometimes it is more natural to reason about recursion equations than to argue about the fixed point operator (fix). This is particularly true if "simultaneous" recursion is involved. Therefore, we introduce the following notation as an alternate for the fixed point operator.

A term of the form

$$\begin{pmatrix} x_1 \\ x_2 \\ \cdots \\ \cdots \\ x_n \end{pmatrix} \leftarrow \begin{pmatrix} T_1(x_1, \ldots, x_n) \\ T_2(x_1, \ldots, x_n) \\ \cdots \\ \cdots \\ T_n(x_1, \ldots, x_n) \end{pmatrix}$$

where T_i are arbitrary terms that may involve x_j's is equivalent to

$$(fix\ \lambda x.\langle T_1(x^{\#1}, \ldots, x^{\#n}), \ldots, T_n(x^{\#1}, \ldots, x^{\#n})\rangle)^{\#1}$$

The following syntactic operations can easily be shown to be sound.

- Row introduction — If terms T_i contain a term T then several occurrences of T can be replaced by a new variable x_{n+1} if the new row $x_{n+1} \leftarrow T$ is added to the system.[5]
- Elimination of redundant rows — If $x_i, i \neq 1$ does not occur in any term except possibly in T_i, then row i can be deleted from the system.
- Elimination of duplicate rows — If two rows i and j, $i, j \neq 1$ are identical up to a renaming of the variables x_i and x_j, then one of these rows, say i, can be deleted if all occurences of x_i are replaced by x_j in the remaining system.
- Substitution — Given a row $x \leftarrow T$, then any number of occurences of x in any T_i may be replaced by T.

We use these rules to prove equalities of the form $fix\ f = fix\ g$ by rewriting both fixed points as recursion equations and transforming them into isomorphic

5.) observing some obvious restrictions concerning free and bound variables

equations. Although this proof technique is useful in many cases it is by no means complete; in general fixed point induction is required.

3. Denotational semantics

Techniques and formalism to define syntactic aspects of programming languages are fairly well understood. Denotational semantics, however, is not yet common knowledge. Therefore we review the basic principles at this point. Semantics of the source and target language will be defined denotationally.

3.1. General concepts

A denotational language definition is a mapping which assigns a "meaning" to each program.[6]

The next question of course is, what is the meaning of a program. The method leaves ample room for different meanings. In general, the meaning of a program is a mapping from input data to a set of answers. Answers in turn can be of different flavour. For example, an answer could be an output file or a runtime error. Alternatively, answers could be *true* and *false* merely indicating termination or nontermination.

Note, that a denotational definition is not an interpreter. The meaning of a program is a (function) object which can be reasoned about. The meaning of a program is purely extensional. This means that two programs are considered equal whenever they denote the same function. For example, questions as to the complexity of a program or order of evaluation cannot be answered on the basis of a denotational definition. This in turn has important advantages. In particular in the case of a compiler it leaves ample room for optimizations. As indicated in the introduction, we will prove that source and target program have the same meaning. Thus, sophisticated code optimization could be incorporated in our compiler without having to change the formal language definitions.

One part of a denotational definition is the definition of the underlying domains. By specifying the domains we define the relevant semantic concepts and their relationship. For example, such concepts as types, storable values, locations, answers etc. are specified.

A set of "valuation functions" assigns a meaning to language constructs in terms of the semantic concepts.

The comparison to first order logic may help clarify the situation (section 1.2). Formulas are syntactic objects. An interpretation assigns a meaning (a truthvalue

6.) A program denotes its meaning, hence the name denotational semantics.

in this case) to each formula. Semantic domains are fairly simple, we only have one universe. But this need not be so, just consider many sorted logical systems.

3.2. Semantic concepts of Algol–like languages

Imperative languages such as Pascal and Algol are an outgrowth of the vonNeuman architecture. Therefore their semantics is most naturally described in terms of *locations, values* stored in loacations and a *memory*, here called *store*. Let L and V be the domains of locations and values respectively. A store can then be defined as a mapping from L to V, i.e. we write $S = L \to V$; it assigns a value to each location.

Another concept is that of identifiers used in a program. Identifiers name variables, procedures, functions and the like. An *environment* is used to keep track of all the current definitions of identifiers. Considering only variables for the time being, an environment is a mapping which assigns a location to each variable identifiers; we write $U = Id \to L$. In practice environments are more complicated.

What is the meaning (value) of a statement? A naive answer is, that a statement is a mapping from stores to stores; a statement maps the store before its execution into the store after its execution. However, this approach is inadequate to define the meaning of statements that do not normally terminate or alter the "normal" controll flow with a jump. Consider the two statements $\Gamma_1;\Gamma_2$. We would like to describe the meaning of this sequence of two statements as the composition of their individual meaning. Assume that Γ_1 and Γ_2 are defined by a mappings $f_1, f_2 \in S \to S$ respectively. The meaing of $\Gamma_1;\Gamma_2$ would be the mapping $f_2 \circ f_1$. Suppose Γ_1 does not terminate normally but instead produces a runtime error or a jump to another part of the program; our simple model clearly fails: f_2 will not be applied to the result of f_1 since controll never reaches Γ_2 in this case.

To remedy this situation *continuations* have been introduced [SW74]; they are based on an idea derived from "tailrecursion" [Ma71]. A continuation is a function from stores into answers, i.e. $C = S \to A$. At a given point during program execution a continutation specifies the behaviour of the remainder of the program. This behaviour is a mapping of the current store to the final result (answer) of the program. The way to describe the meaning of statements now is as follows. Suppose we know the continuation that obtains after a statement Γ, that is we know what the answer of the program will be once Γ terminated with a given store. Now we can determine the continuation that obtains before the execution of Γ. Thus the meaning of statements will be a mapping from continuations to continuations $(C \to C)$. But note, that the reasoning proceeds backwards, i.e. the meaning of a statement Γ maps the continuation "after" Γ into that "before" Γ.

Using continuations the description of runtime errors and jumps poses no

problem at all. Consider the sequence $x \leftarrow a/0;\Gamma$. The meaning of $x \leftarrow a/0$ will be $\lambda\theta\sigma.divide_error$. That is, whatever continuation θ holds before Γ and after $x \leftarrow a/0$, the continuation before $x \leftarrow a/0$ will be $\lambda\sigma.divide_error$. I.e. no matter what state we are in when we execute $x \leftarrow a/0$, the answer of the program is *divide_error*.

A similar situation exists for expressions. A naive view is to consider an expression as a mapping from stores to values. Again, this would not capture side effects of expressions, runtime errors, and jumps out of expressions. The solution is the introduction of expression continuations $K = V \rightarrow C$. Intuitively, an expression continuation tells us what the final answer of the programm will be given a value (the result of an expression) and a store; it tells us what "to do" with the expression value if we succeed in computing it. The meaning of an expression then is a mapping from expression continuations to continuations. Given the expression E and the expression continuation κ and suppose that E evaluates to ϵ, then the resulting continuation will be $\kappa\epsilon$. On the other hand, if E causes a runtime error, the continuation will be $\lambda\sigma.error$.

In a similar fashion more complex domains are constructed; examples are function and procedure values. For example, a procedure is a mapping in $V^* \rightarrow C \rightarrow C$, i.e. it takes a list of parameter values and a continuation and produces a new continuation.

Continuations, expression continuations, procedure, and function values all are domains with a similar structure. In the definition of *LS* we introduce the novel concept of generalized continuations which combines all of the above domains into one and leads to some simplification of the language definition (see chapter III, 5.1.1).

3.3. Denotational definition of a machine language

Concepts used to define high level languages can also be used to define machine languages. Since we have no variable names the environment is significantly simpler. In chapter III we describe a stack machine and define appropriate domains. There will be stacks S, memories M, and displays D. Continuations in the target language are mappings of the form $D \rightarrow S \rightarrow M \rightarrow A$. The intuitive meaning of such a continuation is similar to that in the source language case: given a display, a stack, and a memory, the continuation yields the final answer of the program.

3.4. Notational issues

A main criticism of denotational semantics is that the notation is too complicated and confusing. We have adopted the notation introduced initially by Scott

and Strachey [SS71] and feel that it is very appropriate.

Since the formulas we are dealing with are extremely large, a short notation is imperative for comprehension and overview. Definitions and proofs are more transparent and smaller. As a trade off we have to learn and get familiar with the new notation.

The main points of our notation can be summarized as follows. Function application is written as juxtaposition as in $f\ y$. The semicolon is used to break the normal precedence of function application as in $f\ x; g\ z$. Special parentheses $[\![\ldots]\!]$ are introduced to distinguish syntactic from semantics objects. Attempts are made to have variables named by single letters; we use of roman, greek, and script fonts in upper and lower case.

Mathematics has produced plenty of examples where new short notations have been fruitful. Calculus without $\int$, $\oint$, $\frac{\partial f}{\partial x}$ is unthinkable.

Examples of denotational definitions written in different notations are not very readable, more error prone, and of less help in reasoning about the language [DK79, Pa79, BJ78].

4. Verification techniques

We assume the reader to be familiar with the principles of Hoare's logic [Ho69] as well as the use of automated verification systems, such as the Stanford verifier [IL75, SV79].

This section deals with three problem areas relevant for compiler verification which are not too well understood yet. We introduce new techniques to deal with pointers, quantification, and axiomatization of Scottery.

In general reasoning about pointers is extremely tedious. To isolate special simple cases we classify pointer operations as "reading", "updating", and "extension" operations; we prove a theorem that greatly simplifies reasoning about extension operations.

The second problem is that of expressing quantified formulas in a quantifier free assertion language. We present a deduction step in Hoare's logic called "weak $\forall$–introduction", comparable to $\forall$–introduction in first order logic.

Thirdly, we discuss the axiomatization of Scott's computable functions. This requires a satisfactory treatment of the undefined element ($\perp$) as well as higher order functions.

4.1. Pointers

4.1.1. Reference classes

A satisfactory formal treatment of pointers has first been proposed by Luckham and Suzuki in [LS76]. The basic idea is to associate a "reference class" with each pointer type. The reference class is the collection of data objects pointed to by pointers of this type. The meaning of data structures involving pointers is explained relative to a reference class. The operation of dereferencing is described as "indexing" the reference class with a pointer. In fact, the relation of pointers and reference classes is very much like that of indices and arrays.

Formally the expression $p\uparrow$ means $r \subset p \supset$ for the reference class r associated with the type of p. Assignments to data objects pointed to by pointers, e.g. $p\uparrow \leftarrow e$, are described as assignments to a whole reference class, e.g. $r \leftarrow < r, \subset p \supset, e >$

Similar to the array term $< a, [i], e >$ the term $< r, \subset p \supset, e >$ denotes the reference class r after $r \subset p \supset$ has been replaced by e. Assignment and selection obey the well known axioms

$$< r, \subset p \supset, e > \subset q \supset = \textbf{if } p = q \textbf{ then } e \textbf{ else } r \subset q \supset$$
$$< r, \subset p \supset, r \subset p \supset > = r$$

Reference classes are not part of Pascal[7]; the Stanford verifier introduces a reference class $\#t$ automatically whenever the program contains a type declaration of the form **type** $pt = \uparrow t$.

In addition to assignment and selection the "extension" operation $r \bigcup p$ is defined for reference classes. This construct is used to describe the effect of generating a *new* data object; it extends an existing reference class r by the new cell $p\uparrow$ pointed to by p.

4.1.2. Reasoning about pointers

Suppose we are given a reference class r and a pointer p pointing to the beginning of a linked list. If we now add a new element to this list for example, then this will change the reference class r. A central problem in reasoning about reference classes is to prove that this change does not destroy the already existing list structure. Proof techniques for this case have been outlined in [LS75]. However, these techniques are fairly tedious and the resulting complexity is unacceptable for our compiler proof.

7.) A similar class construct was part of the original Pascal definition [Wi71]; also, reference classes, called collections, are included in Euclid [Lo78]

We need more abstract operations on pointers that are easier to deal with. An obvious idea is to use the concept of data abstraction [Gu75, GH76] to encapsulate pointer operations and define modules which provide abstract list or tree operations. Still, the implementation of such a module has to proceed on a very low level and poses all of the above problems.

Another approach has been proposed recently [Su80]. Suzuki introduces the idea of pointer rotations. However, this method is not complete in the sense that all pointer operations could be expressed as rotations. Also, proof techniques for dealing with pointer rotations have yet to be developed.

Our approach to pointers is as follows. We distinguish three different operations on pointer structures, "reading", "updating", and "extending". Special proof techniques for these three cases are described below.

"Reading" data stored in a pointer structure (e.g. linked list) causes no problem whatsoever; no change of reference classes occurs and thus all structures are preserved.

"Updating" on the other hand is extremely difficult to handle. An update operation is one which changes an existing reference class. The situation is familiar to the lisp programmer using language features like RPLACA and RPLACD. We show in section 4.3.5 that a disciplined use of update operations can be adequately described by least fixed points. Update operations are only used at one place throughout the compiler implementation.

We talk about "extending" a pointer structure when all changes are of the form:

- create a new cell in the reference class
- change this new cell.

Reasoning about extension operations is substantially simpler than that about update operations. This situation is analysed in the following subsection.

4.1.3. Reasoning about extensions

Let **type** $pt = \uparrow t$ with associated reference class $\#t$. Now consider a procedure *extend* which changes $\#t$ by extension. Clearly, $\#t$ is a **var** *global* to *extend* as in

procedure *extend*($p : pt$);
global(**var** $\#t$);
entry ...

Let $\#t_0$ and $\#t_f$ be the value of $\#t$ before and after a call to *extend*. If $\#t_f$ is an extension of $\#t_0$ then all elements of $\#t_0$ are unchanged in $\#t_f$. Thus, any abstract object represented by $\#t_0$, for example *listrep*($p, \#t$), is not changed by

the extension, i.e.

$$listrep(p, \#t_0) = listrep(p, \#t_f)$$

Consequently, any proposition on abstract objects represented by a reference class are invariant to extension operations:

$$P(listrep(p, \#t_0)) => P(listrep(p, \#t_f)).$$

Note, that predicates on the reference class itself are in general not invariant (e.g. the cardinality of a reference class will be changed by an extension operation). Thus, if we only allow propositions on the abstract objects represented by a pointer structure we may consider the reference class $\#t$ a constant rather than a variable global to extension operations.

The above reasoning can be formalized by suitable assertions and the result of this reasoning can be proved as part of the verification. We demonstrate this in the proof of the parser in chapter IV. The above concepts are expressed in the assertion language using predicates *subclass* and *proper*. $subclass(\#t_1, \#t_2)$ is true, if $\#t2$ is an extension of $\#t_1$. *proper* is a predicate on abstract objects represented by reference classes. It is true if such an object is well defined, i.e. if the representation function does not dereference an undefined pointer $\#t \subset p \supset$ where the cell pointed to by p has not been added to $\#t$.

In the verification of the semantic analysis we do not repeat this reasoning, rather we resort to the meta theorem that allows us to consider a reference class unchanged by a procedure if the new reference class is a mere extension of the original reference class.

This reasoning simplifies proofs considerably and lets us concentrate on more relevant things.

4.2. Quantification

The Stanford Verifier, used to mechanize the proofs of our compiler, limits the assertion language to quantifier free first order logic. This has certain computational advantages in the system's theorem prover but in some cases turns out to be a severe limitation. Since we believe that similar problems may arise with other mechanical verifier we describe here two principal methods for dealing with quantification.

A simple example illustrates the situation. Let us assume that we have a procedure $p(y)$ for which we need the exit condition $\forall x.R(x, y)$. One solution is to introduce a new predicate symbol $RQ(y)$ with the interpretation $RQ(y) \equiv \forall x.R(x, y)$. This method is general and allows elimination of all existential and universal quantifiers in abitrary situations. The main disadvantage is, that all

theorems about R that are only provable in a full first order theory have to be proven outside the verifier and supplied as lemmas about RQ.

In many cases a more elegant solution is possible which allows much of the proof to be handled by the verifier. Suppose that for a particular call to p we know that only one certain instance of the exit condition is used for the proof of this call, i.e. the exit condition $\forall x.R(x,y)$ is particularized to $R(a,y)$ in the proof of this call. In this case we can write a procedure $\acute{p}(x,y)$ with exit condition $R(x,y)$. The additional virtual parameter x is used to provide the correct instance of the exit condition; e.g. if a particular call requires the instance $R(a,y)$ of the exit condition for its proof, then we supply the virtual parameter a in this call. Note, that for this method no new predicates and lemmas are necessary.

This second method does not immediately apply for cases where we actually need the full quantification in the exit assertion. However, the following theorem allows us to verify a procedure in quantifier free logic using virtual parameters and then introduce quantifiers later on. This process is the analogue of the well known $\forall$-introduction in first order logic applied to Hoare's weak program logic.

Suppose we are given a procedure $p(x_1,\ldots,x_n,v_1,\ldots,v_m)$ where all v_i are virtual parameters. Suppose further, that we have a proof for the body S of p with the specifications

$$\{Q(x_1,\ldots,x_n,v_1,\ldots,v_m)\}S\{R(x_1,\ldots,x_n,v_1,\ldots,v_m)\}$$

Clearly, the final values of the var parameters among the x_i do not depend on the initial values of the virtual parameters v_i. Now we can construct a procedure $\acute{p}(x_1,\ldots,x_n)$ with body $\acute{S}$ in which we omit all virtual code.

Theorem [weak $\forall$–introduction] Whenever

$$\{Q(x_1,\ldots,x_n,v_1,\ldots,v_m)\}S\{R(x_1,\ldots,x_n,v_1,\ldots,v_m)\}$$

then it is true that

$$\{x_i = \acute{x}_i\}\acute{S}\{\forall v_i.Q(\acute{x}_1,\ldots,\acute{x}_n,v_1,\ldots,v_m) => R(x_1,\ldots,x_n,v_1,\ldots,v_m)\}.$$

Proof Suppose the theorem were false, i.e. let x_i be such that $\acute{S}$ terminates and assume that there is a tuple v_i with

$$Q(\acute{x}_1,\ldots,\acute{x}_n,v_1,\ldots,v_m) \wedge \neg R(x_1,\ldots,x_n,v_1,\ldots,v_m).$$

Clearly, the entry condition for the original procedure body S is satisfied and S will also terminate for x_i because termination must be independant of the values of

virtual variables. But now by

$$Q(x_1, \ldots, x_n, v_1, \ldots, v_m)\{S\}R(x_1, \ldots, x_n, v_1, \ldots, v_m)$$

$R(x_1, \ldots, x_n, v_1, \ldots, v_m)$ must be true which contradicts our assumption.

Of course, we have to be able to express the new exit condition

$$\forall v_i.Q(\acute{x}_1, \ldots, \acute{x}_n, v_1, \ldots, v_m) => R(x_1, \ldots, x_n, v_1, \ldots, v_m)$$

in the assertion language. But this can be done by using the first method and introducing a new predicate symbol, say $QR(\acute{x}_1, \ldots, \acute{x}_n, x_1, \ldots, x_n)$. We still retain the advantage that for the proof of this exit condition we only need the definitions and properties of Q and R and that this proof is done in a quantifier free logic. To make any use of the new exit assertion QR, however, we need the theorem

$$\begin{aligned} &QR(\acute{x}_1, \ldots, \acute{x}_n, x_1, \ldots, x_n) \wedge \\ &Q(\acute{x}_1, \ldots, \acute{x}_n, v_1, \ldots, v_m) \\ &\quad => R(x_1, \ldots, x_n, v_1, \ldots, v_m) \end{aligned}$$

Application of this method is shown in chapter IV when we discuss code generation.

4.3. Computable functions and first order logic

The language to be implemented is formally defined using Scott's logic of computable functions, the implementation language is Pascal, and program proofs will be expressed in a first order assertion language.

In this section we discuss, how these three different formalism can be tied together. First, we show how abstract objects of domains can be represented as Pascal data objects. Second, we discuss how the formal language definition can be axiomatized in a first order assertion language. Finally, we present a technique to compute the Pascal representation of least fixed points.

4.3.1. Representations

We systematically distinguish *concrete* types in the programming language from *abstract* domains in the compiler definition. The relation between abstract and concrete is established through *representation functions* mapping objects of the program into abstract objects in the definitional language.

For the following discussion a type is either a Pascal type or a tuple of Pascal types, written $T_1, \times \ldots \times T_n$.

A domain D is represented by a type T if there is a representation function $r \in T \mapsto D$ To emphasize that such a function serves as representation function we write $r \in T \leftrightsquigarrow D$; i.e. semantically $\leftrightsquigarrow$ is equivalent with $\mapsto$.

In a particular situation we may not be interested in representing all elements of a domain. Similarly, we allow different representations of the same element of a domain. Thus, in general, a representation function can be arbitrary; we require neither surjectivity nor injectivity.

We now give several examples of types representing domains. More complicated representations will be introduced in the individual program proofs.

- A scalar type T represents the flat domain $T_\perp$ (up to isomorphism). The representation function $Trep \in T \leftrightsquigarrow T_\perp$ is a simple embedding.

  For example, we have $truthrep \in boolean \leftrightsquigarrow \{TT, FF\}_\perp$ with

  $$truthrep(true) = TT, \quad truthrep(false) = FF.$$

- A pointer type $P = \uparrow T$ is the set of infinitely many pointers p_i together with the element NIL. Let $D_P = \{p_1, \ldots, p_n, \ldots\}_\perp$; we have $rep_P \in P \leftrightsquigarrow D_P$ where $rep_P = \lambda p.$ **if** $p = NIL$ **then** $\perp$ **else** p. In this case we have a representation of $\perp$.
- A product domain $D = D_1 \times \ldots \times D_n$ can be represented by $T =$ **record** $I_1 : T_1; \ldots; I_n : T_n$ **end** with $rep \in T \leftrightsquigarrow D$ as

  $$rep(x) = \langle rep_1(x.I_1), \ldots, rep_n(x.I_n) \rangle$$

  where $rep_i \in T_i \leftrightsquigarrow D_i$.
- A sum $D = D_1 + \ldots + D_n$ can be represented by the record[8]

  $$T = \textbf{record } tag : 1..n;\ I_1 : T_1; \ldots; I_n : T_n \textbf{ end}.$$

  The representation function $rep \in T \leftrightsquigarrow D$ is defined as $rep = \lambda x.$ **let** $i = x.tag$ **in** $rep_i(x.I_i) : D$ where $rep_i \in T_i \leftrightsquigarrow D_i$.

Let T be a scalar type and $rep \in T \leftrightsquigarrow T_\perp$ as above, then there is a function rep^{-1} such that $proper\ \epsilon \Rightarrow rep(rep^{-1}\epsilon) = \epsilon$.

- For a flat domain D_1 the function domain $D = D_1 \to D_2$ can be represented by an array type $T = array[T_1]of\ T_2$. We use $rep \in T \leftrightsquigarrow D$

  $$rep(a) = \lambda \epsilon \in D_1.\ \textbf{if } proper\ \epsilon \textbf{ then } rep_2(a[rep_1^{-1}(\epsilon)]) \textbf{ else } \perp$$

  where $rep_1 \in T_1 \leftrightsquigarrow D_1$ and $rep_2 \in T_2 \leftrightsquigarrow D_2$.

8.) Variant records would give a more natural representation; however, the Stanford verifier does not allow records with variants

A reference class can be considered an array indexed by pointers. Consequently, $\#T$ can be used to represent $D_P \to D$ where D_P is the domain represented by the pointer type $\uparrow T$ and D is represented by the T.

- A reflexive domain $D = A + (B \times D)$ can be represented as follows. Let $rep_A \in T_A \nwarrow\!\!\searrow A$ and $rep_B \in T_B \nwarrow\!\!\searrow B$. D is represented by $\uparrow T \times \#T$, where

$$\textbf{type } T = \textbf{record } tag : 1..2;\ a : T_A;\ b : T_B;\ c : \uparrow T \textbf{ end}$$

i.e. $rep \in T \nwarrow\!\!\searrow A + (B \times D_P)$. We define $recrep \in (\uparrow T \times \#T) \to D$ as

$$recrep(p, r) = (fix\lambda f.\lambda p.\ \textbf{if } r \subset p \supset .tag = 1 \textbf{ then } rep_A(r \subset p \supset .a) \textbf{ else } \langle rep_B(r \subset p \supset .b), f(r \subset p \supset .c)\rangle)p\ .$$

4.3.2. Standard interpretations

An obvious observation is that the usual axiomatizations of well known theories (e.g. Presburger arithmetic) do not allow corresponding functions from Scott's logic as models (e.g. addition over the flat domain of integers). For example, the sentence $\forall x.x \neq x + 1$ is true for conventional addition but false for the flat integer CPO $N_\perp$ and $+ \in N_\perp \to N_\perp \to N_\perp$.

This problem is relevant for our work since a first order theorem prover may know (interpret) the symbol $+$. The solution is immediate using the notion of representation introduced above. We distinguish the type integer from the flat integer cpo. The symbol $+$ is reserved for conventional addition on integers; we introduce a new function *plus* to denote addition over the flat integer cpo $N_\perp$.

plus can be defined axiomatically as

$$\begin{aligned} plus(x, \perp) &= \perp \\ plus(\perp, y) &= \perp \\ plus(intrep(x), intrep(y)) &= intrep(x + y) \end{aligned}$$

where *intrep* is a representation function mapping integers into the integer cpo. With these axioms the equation

$$x = plus(x, intrep(1))$$

does not lead to inconsistencies. Rather it only allows to deduce

$$\neg\exists y.x = intrep(y).$$

4.3.3. Higher order functions

In Scott's logic we have to deal with arbitrary high order function (in fact, elements of recursively defined domains can be considered "infinite" order functions

[Sc72a]). The question is, how we can describe these objects in first order logic. The straightforward solution is to simply consider functions as values. All operations on functions, including function application have to be introduced as functions in the logic.

In addition we have to be able to talk about least fixed points of these functions. One solution is to axiomatize the ordering "less defined than" ($\sqsubseteq$). Given this ordering it can be defined what it means to be the least fixed point etc. The main problem with this approach is, that things become excessively messy and incomprehensible and we should look for a simpler solution.

In [CM79] Cartwright and McCarthy propose a *minimization schema* to capture the idea of the least fixed point. But this approach is limited to first order functions and by introducing the predicate *isD* (to test whether an element is proper) implicitely introduces Scott's ordering ($\sqsubseteq$) for the special case of flat domains.

Here we go a different way. Given a denotational definition with a fixed set of domains. We say that a domain D is *terminal* if there is no other domain $\acute{D}$ defined in terms of D. In particular, elements from a terminal domain can never appear as arguments to functions (otherwise the function would be an element of a domain defined in terms of this terminal domain).

For a given definition we let U be the sum of all non-terminal domains. All objects in U are axiomatized as values (U is the universe of the standard interpretation). Terminal functions from $U \rightarrow U$ are first order functions. By the definition of terminal domains there are no higher order functions.

If functions are axiomatized as values we need an operation denoting function application. We use the symbol $\otimes$ for this purpose; i.e. the standard interpretation for $\otimes$ is $\mathcal{T}[\![x \otimes y]\!]\phi = (\mathcal{T}[\![x]\!]\phi)(\mathcal{T}[\![y]\!]\phi)$.

With these tools we can axiomatize non-recursive function definitions in a straightforward way. For example suppose we have a first order term $\acute{T}(x)$ representing the term $T(x)$ in Scottery. A definiton $g = \lambda\ x.T(x)$ is translated into the axiom $g \otimes x = \acute{T}(x)$.

4.3.4. Least fixed points

For the above axiomatization of Scottery to be of any use we have to be able to axiomatize recursively defined functions and least fixed points. Suppose we are given $f = fix(\lambda\ x.T(x))$ and $\acute{T}$ as above. For f we can write the axiom $\acute{T}(f) = f$ which is clearly satisfied by $fix(\lambda x.T(x))$. But the axiom merely asserts that f is a fixed point. It does not require that f be minimal: any fixed point satisfied the above axiom. We say that f is *weakly* axiomatized.

In many cases a weak axiomatization suffices to prove relevant properties of f.

Suppose we can give a proof in first order logic of the form

$$f = g \otimes f \vdash P(f).$$

This means that P is true for all interpretations of f satisfying $f = g\ f$. In particular $P(fix\ g)$ is true. Although weak axiomatization is a useful device it is not sufficient in general and many properties P cannot be proven.

Since $fix \in (D \to D) \to D$ is a function we have the obvious property

$$f = g \Rightarrow fix\ f = fix\ g.$$

Again, a useful way of proving properties of fixed points.

4.3.5. Operationalization of fixed points

In the compiler to be written it is necessary to compute least fixed points. For example, in the source *LS* one can define a type which is a pointer to a type which is a pointer to a type which is a pointer ... etc. The semantics of such a recursive type is given as a least fixed point. In order to do type–checking in the compiler we have to compute a data object which represents this least fixed point. The weak axiomatization introduced in 4.3.4 is insufficient to prove that a particular computation computes the "least" fixed point.

The solution presented below uses the representation theory as a way out: instead of axiomatizing a least fixed point we axiomatize a representation of this fixed point.

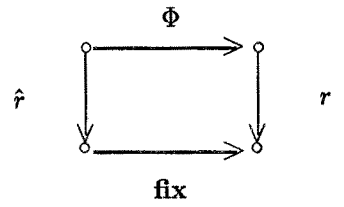
As a simple example consider the domain $D = A + (B \times D)$ for which we defined a representation *recrep* in section 4.3.1 using

$$\textbf{type } T = \textbf{record } tag : 1..2;\ a : T_A;\ b : T_B;\ c :\uparrow T \textbf{ end}$$

Let $x \in T$ be the record $[2, ?, b, p]$. With the assignment $p \uparrow\leftarrow x$ we create the reference class $\langle r, \subset p \supset, [2, ?, b, p]\rangle$. If we now compute $recrep(p, \langle r, \subset p \supset, [2, ?, b, p]\rangle)$ we get

$$\begin{aligned}&(fix\lambda f.\lambda p.\ \textbf{if } [2, ?, b, p].tag = 1 \textbf{ then } rep_A(?) \textbf{ else } \langle rep_B(b), f(p)\rangle)p\\ &= (fix\lambda f.\lambda p.\langle rep_B(b), f(p)\rangle)p\\ &= fix\lambda x.\langle rep_B(b), x\rangle\ .\end{aligned}$$

Thus, we have shown that the concrete assignment $p \uparrow\leftarrow x$ creates a data object that represents a least fixed point. We call this process "operationalization" of fixed points. The general situation can be characterized as follows. For a domain D we define a representation $r \in T \leftrightsquigarrow D$ as well as a representation $\hat{r} \in \hat{T} \leftrightsquigarrow D \to D$. We then define an operation Φ on $\hat{T}$, producing an object in T such that $r(\Phi\hat{t}) = fix(\hat{r}\hat{t})$; i.e. the following diagram commutes:



We now prove a theorem for the computation of representation of fixed points for a certain class of domains. Similar techniques may be proven for arbitrary domains, however, the class considered here is sufficient for the compiler proof.

The domains we consider are those freely generated by a set of functions g_i. More precisely, let g_i be n-ary function symbols; we consider the domain D of terms constructed from g_i and the symbol $\perp$[9].

Using techniques described above D can be represented as follws. Let $T = N \times P \times \ldots \times P$ where $P = \uparrow T$; let $\#T$ be the reference class for P. Now D is represented by pairs in $\#T \times P$ by means of $r \in \#T \to P \leftrightsquigarrow D$ as

$$r\ c\ p = \big(\ \textbf{fix}\ \ \lambda h.\lambda x.\ \textbf{let}\ \langle i, p_1, \ldots, p_n\rangle = c\ x\ \textbf{in}\ \textbf{let}\ y_j = h\ p_j\ \textbf{in}\ g_i(y_1, \ldots, y_n)\big)p$$

Furthermore we can define a representation for the domain $D \to D$ in terms of triples $\#T \times P \times P$ with $\hat{r} \in \#T \to P \to P \leftrightsquigarrow D \to D$ as

$$\hat{r}\ c\ p\ q = \big(\ \textbf{fix}\ \ \lambda f.\lambda x.\lambda d.\ \textbf{let}\ \langle i, p_1, \ldots, p_n\rangle = c\ x\ \textbf{in}\ \textbf{let}\ y_j = \textbf{if}\ x = q\ \textbf{then}\ c\ p\ \textbf{else}\ f\ p_j\ d\ \textbf{in}\ g_i(y_1, \ldots, y_n)\big)p$$

Clearly, not all functions in $D \to D$ can be represented in this way but many useful ones can (recall that we do not require representations to be surjective). We now have the important

Theorem For r and $\hat{r}$ as defined above:

$$r\ (c[c\ p/q])\ p = \ \textbf{fix}\ (\hat{r}\ c\ p\ q)$$

Proof We express the least fixed points as recursion equations as introduced in

9.) D is the "free algebra" or "Herbrand universe" generated by g_i

section 2.4.2. Observing

$$r\,(c[c\,p/q])\,p = (\,\mathbf{fix}\ \ \lambda h.\lambda x.\\ \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = \mathbf{if}\ x = q\ \mathbf{then}\ c\,p\ \mathbf{else}\ c\,x\ \mathbf{in}\\ \mathbf{let}\ y_j = h\,p_j\ \mathbf{in}\ g_i(y_1, \ldots, y_n))p$$

the left hand side of the theorem becomes

$$\begin{pmatrix} z_1 \\ h \end{pmatrix} \leftarrow \begin{pmatrix} h\,p \\ \lambda x.\ \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = \mathbf{if}\ x = q\ \mathbf{then}\ c\,p\ \mathbf{else}\ c\,x\ \mathbf{in} \\ \qquad \mathbf{let}\ y_j = h\,p_j\ \mathbf{in}\ g_i(y_1, \ldots, y_n) \end{pmatrix}$$

The right hand side may be rewritten as

$$\begin{pmatrix} z_2 \\ f \end{pmatrix} \leftarrow \begin{pmatrix} f\,p\,z_2 \\ \lambda x.\ \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = c\,x\ \mathbf{in} \\ \qquad \mathbf{let}\ y_j = \mathbf{if}\ p_j = q\ \mathbf{then}\ z_2\ \mathbf{else}\ f\,p_j\,z_2\ \mathbf{in}\ g_i(y_1, \ldots, y_n) \end{pmatrix}$$

Let us introduce $k = \lambda p.f\ p\ z_1$ in this system yielding

$$\begin{pmatrix} z_2 \\ k \end{pmatrix} \leftarrow \begin{pmatrix} k\,p \\ \lambda x.\ \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = c\,x\ \mathbf{in} \\ \qquad \mathbf{let}\ y_j = \mathbf{if}\ p_j = q\ \mathbf{then}\ k\,p\ \mathbf{else}\ k\,p_j\ \mathbf{in}\ g_i(y_1, \ldots, y_n) \end{pmatrix}$$

Now let $t\,p_j = \mathbf{if}\ p_j = q\ \mathbf{then}\ k\,p\ \mathbf{else}\ k\,p_j$ in this system, thus

$$\begin{pmatrix} z_2 \\ t \end{pmatrix} \leftarrow \begin{pmatrix} \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = c\,p\ \mathbf{in}\ \ \mathbf{let}\ y_j = t\,p_j\ \mathbf{in}\ g_i(y_1, \ldots, y_n) \\ \lambda x.\ \mathbf{let}\ \langle i, p_1, \ldots, p_n\rangle = \mathbf{if}\ x = q\ \mathbf{then}\ c\,p\ \mathbf{else}\ c\,x\ \mathbf{in} \\ \qquad \mathbf{let}\ y_j = t\,p_j\ \mathbf{in}\ g_i(y_1, \ldots, y_n) \end{pmatrix}$$

which proves the equality.